MR-Coupler: Automated Metamorphic Test Generation via Functional Coupling Analysis

CONGYING XU, The Hong Kong University of Science and Technology, China and Guangzhou HKUST Fok Ying Tung Research Institute, China
HENGCHENG ZHU, The Hong Kong University of Science and Technology, China and Guangzhou HKUST Fok Ying Tung Research Institute, China
SONGQIANG CHEN, The Hong Kong University of Science and Technology, China and Guangzhou HKUST Fok Ying Tung Research Institute, China
JIARONG WU, The Hong Kong University of Science and Technology, China and Guangzhou HKUST Fok Ying Tung Research Institute, China
VALERIO TERRAGNI, University of Auckland, New Zealand
SHING-CHI CHEUNG*, The Hong Kong University of Science and Technology, China and Guangzhou HKUST Fok Ying Tung Research Institute, China

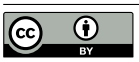
Metamorphic testing (MT) is a widely recognized technique for alleviating the oracle problem in software testing. However, its adoption is hindered by the difficulty of constructing effective metamorphic relations (MRs), which often require domain-specific or hard-to-obtain knowledge. In this work, we propose a novel approach that leverages the functional coupling between methods, which is readily available in source code, to automatically construct MRs and generate metamorphic test cases (MTCs). Our technique, MR-Coupler, identifies functionally coupled method pairs, employs large language models to generate candidate MTCs, and validates them through test amplification and mutation analysis. In particular, we leverage three functional coupling features to avoid expensive enumeration of possible method pairs, and a novel validation mechanism to reduce false alarms. Our evaluation of MR-Coupler on 100 human-written MTCs and 50 real-world bugs shows that it generates valid MTCs for over 90% of tasks, improves valid MTC generation by 64.90%, and reduces false alarms by 36.56% compared to baselines. Furthermore, the MTCs generated by MR-Coupler detect 44% of the real bugs. Our results highlight the effectiveness of leveraging functional coupling for automated MR construction and the potential of MR-Coupler to facilitate the adoption of MT in practice. We also released the tool and experimental data to support future research.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: Metamorphic Testing, Large Language Models, Functional Coupling

*Shing-Chi Cheung is the corresponding author.

Authors' Contact Information: Congying Xu, The Hong Kong University of Science and Technology, Hong Kong, China and Guangzhou HKUST Fok Ying Tung Research Institute, Guangzhou, China, cxubl@connect.ust.hk; Hengcheng Zhu, The Hong Kong University of Science and Technology, Hong Kong, China and Guangzhou HKUST Fok Ying Tung Research Institute, Guangzhou, China, hzhuaq@connect.ust.hk; Songqiang Chen, The Hong Kong University of Science and Technology, Hong Kong, China and Guangzhou HKUST Fok Ying Tung Research Institute, Guangzhou, China, i9s.chen@connect.ust.hk; Jiarong Wu, The Hong Kong University of Science and Technology, Hong Kong, China and Guangzhou HKUST Fok Ying Tung Research Institute, Guangzhou, China, jwubf@cse.ust.hk; Valerio Terragni, University of Auckland, Auckland, New Zealand, v.terragni@auckland.ac.nz; Shing-Chi Cheung, The Hong Kong University of Science and Technology, Hong Kong, China and Guangzhou HKUST Fok Ying Tung Research Institute, Guangzhou, China, scc@cse.ust.hk.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/7-ARTFSE206
https://doi.org/10.1145/3808213

ACM Reference Format:
Congying Xu, Hengcheng Zhu, Songqiang Chen, Jiarong Wu, Valerio Terragni, and Shing-Chi Cheung. 2026. MR-Coupler: Automated Metamorphic Test Generation via Functional Coupling Analysis. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE206 (July 2026), 24 pages. https://doi.org/10.1145/3808213

1 Introduction

Metamorphic testing (MT) is a powerful testing technique that helps address the test oracle problem [11, 48]. Specifically, MT tackles this challenge by leveraging relations between multiple outputs (i.e., output relation) given the relations between corresponding inputs (i.e., input relation). In this case, the logical implication from input relation to output relation defines a *metamorphic relation (MR)*. MT has been shown to be effective in various software systems, especially where the expected outputs are difficult to specify (e.g., large language models [14, 15], compilers [30, 51], machine translation [8, 65], question answering systems [64, 69], etc.). One MR can serve as an oracle applied to many test inputs to exercise diverse program behaviors and enhance test adequacy [67].

Despite its potential, the adoption of MT is challenging. A key bottleneck is the construction of effective MRs [48], which requires domain-specific knowledge. Although several attempts have been made to explore the generation of MRs, these approaches suffer from (i) reliance on manual effort [12, 52, 74], (ii) assumptions of regression testing scenarios [4, 5], (iii) restriction to specific domains [33, 34, 49, 71, 72, 74, 78], or (iv) requirements for high-quality specifications [6, 50, 67]. All these studies rely on knowledge that is hard to obtain. Although a recent study [67] reported the possibility of mining the fragmented knowledge required by MRs from test cases, it also found such test cases to be rarely available: they account for only 1% of the studied test cases and are scattered in only 20% of the studied projects. The lack of automatic methodologies for constructing metamorphic test cases (MTCs) hinders the widespread adoption of MT. To ease its adoption, a technique without the above-mentioned limitations is expected. To construct such a technique, *a central challenge is to formulate MRs without relying on knowledge that is hard to obtain.*

Fortunately, we found that the *functional coupling between methods, which is readily available in the code, can be formulated as MRs.* For example, the pair of functions `encrypt` and `decrypt` can formulate an MR $x = decrypt(encrypt(x))$, as shown in Listing 1. This motivates us to formulate MRs by identifying such coupled method pairs. This idea offers several advantages: (1) *readily-available knowledge*: it relies solely on a pair of methods and their implementation, which is by construction available in the scenario of unit testing; (2) *more tractable problem*: this transforms the challenging problem of deriving MRs into code understanding and relation reasoning, which can be effectively handled by current state-of-the-art large language models (LLMs) [35, 53, 66, 70]. For instance, although it is challenging to come up with MRs for a target method `encrypt`, when paired with a coupled method `decrypt`, it becomes easier for LLMs to understand their functionalities separately, realize that they are inverse functions, and then formulate a relation $x = decrypt(encrypt(x))$; (3) *easier bug manifestation*: certain bugs can be revealed more easily with coupled computations. For instance, while it is difficult to reveal the bug in Listing 2 by calling `encrypt` and `decrypt` separately, it becomes easier with the MR $x = decrypt(encrypt(x))$. In summary, leveraging functionally coupled methods as a foundation for MR construction provides a practical and effective pathway to automate metamorphic testing and broaden its applicability.

However, leveraging functionally coupled methods to construct MRs requires addressing two technical issues. First, given a target method, there can be dozens of candidate method pairs, and it is expensive to enumerate all possible method pairs blindly for MR construction. Thus, there is a need for a precise mechanism to identify functionally coupled method pairs, which provides better focal methods for subsequent MR construction. Second, while LLMs enable MTC generation via code understanding and reasoning, the resulting MTCs can be invalid due to hallucination [77].

```java
@Test
public void testEncryptDecrypt() throws Exception {
    String plainText = "Hello AES!";
    SecretKey secKey = AESEncryption.getSecretEncryptionKey();
    // Encrypt the plaintext: invoke on the source input, and produce the source output
    byte[] cipherText = AESEncryption.encryptText(plainText, secKey);
    // Decrypt the ciphertext: invoke on the follow-up input, and produce the follow-up output
    String decryptedText = AESEncryption.decryptText(cipherText, secKey);
    assertEquals(plainText, decryptedText); // output relation assertion
}
```

Listing 1. An MTC that Encodes the MR $x = decrypt(encrypt(x))$ over `encryptText` and `decryptText`

Therefore, we need an effective mechanism to validate the generated MTCs, which allows us to avoid overwhelming developers with false alarms [54].

To tackle these technical issues and effectively generate MTCs, we propose MR-Coupler, an automatic MTC generator for a given target method. It operates in three phases. First, it identifies functionally coupled methods as ingredients for MR construction, based on their signature commonality, functional call, and state interaction. This design is informed by our observation of human-written MTCs and thereby addresses the first technical issue. Second, it employs LLMs to generate MTCs based on each identified functionally coupled method pair by providing relevant and minimal context. Specifically, we instruct LLMs to understand their functionalities and reason about potential MRs between them. To reduce hallucinations that lead to invalid MTCs, we provide an MTC template and retrieve API usages for LLMs to follow. Finally, it validates the generated candidate MTCs via test amplification and mutation analysis. To validate the MTCs without a given ground truth, we create mutants from the original program by injecting artificial faults, and expect more amplified MTCs (from the candidate MTC) to pass on the original version compared with the faulty mutants. This filtering strategy is based on a property of MT: the MR embedded in a correct MTC should apply to many other inputs to effectively kill mutants [67].

We evaluated MR-Coupler on (i) 100 human-written MTCs with corresponding target methods and (ii) 50 real-world bugs as tasks for evaluation. MR-Coupler successfully generates valid MTCs for over 90% of tasks, achieving a 64.90% improvement in valid MTC generation and a 36.56% reduction in false alarms compared with baselines. The MTCs generated by MR-Coupler found 44% of the 50 real bugs. The key components play crucial roles in MR-Coupler: we found that functional coupling between methods guides LLMs to produce valid, bug-revealing MTCs, while MTC amplification and validation steps further improve bug detection and halve false alarms. Last but not least, MR-Coupler can mimic developers in using functionally coupled methods and achieves over 90% MR-skeleton consistency with human-written MTCs, highlighting its potential to assist developers in constructing MTCs. In this paper, we make the following contributions.

- To the best of our knowledge, we are the first to construct MRs by leveraging the functional coupling between methods. Our approach relies solely on the code under test, and thus enables easier MR construction and lowers the barrier to adopting MT.
- We propose MR-Coupler, an automatic approach to generate concrete metamorphic test cases. MR-Coupler instructs LLMs with relevant and minimal context for MR construction, and validates the generated MTCs with a novel filtering mechanism based on mutation analysis.
- We conduct extensive experiments to evaluate the effectiveness of MR-Coupler, including the validity of generated MTCs, the capability of revealing real bugs, and the similarity of generated MTCs to human-written MTCs.
- We make MR-Coupler and the experimental datasets publicly available. The datasets of human-written MTCs and real-world bugs originally discovered by MT are valuable for future research. Our artifact is available at the website of MR-Coupler [57].

```java
public static byte[] encryptText(String plainText, SecretKey secKey) {
    Cipher aesCipher = Cipher.getInstance("AES");
    // aesCipher.init(Cipher.ENCRYPT_MODE, AESEncryption.defaultKey);   // bug
    aesCipher.init(Cipher.ENCRYPT_MODE, secKey);                        // fix
    return aesCipher.doFinal(plainText);
}
public static String decryptText(byte[] byteCipherText, SecretKey secKey) {
    Cipher aesCipher = Cipher.getInstance("AES");
    aesCipher.init(Cipher.DECRYPT_MODE, secKey);
    return aesCipher.doFinal(byteCipherText);
}
```

Listing 2. Code of Methods `encryptText` and `decryptText` [56]

2 Preliminaries

Metamorphic Testing (MT) evaluates a program P using a Metamorphic Relation (MR, $\mathcal{R}$), which is a logical implication from an input relation $\mathcal{R}_i$ to an output relation $\mathcal{R}_o$ [11, 48, 67].

$$\mathcal{R} : \mathcal{R}_i\left(x_s, x_f\right) \implies \mathcal{R}_o\left(x_s, x_f, y_s, y_f\right)$$

$\mathcal{R}_i$ specifies the rule for deriving a *follow-up input* (x_f) from a given *source input* (x_s), while $\mathcal{R}_o$ defines the expected relationship over the inputs and their corresponding outputs (x_s, x_f, y_s, y_f)[1].

A *Metamorphic Test Case (MTC)* is a test case that implements MT. As illustrated in Listing 1, given an MR $\mathcal{R}$ ($x = decrypt(encrypt(x))$) for a target program P (class `AESEncryption`), an MTC has the following steps: (i) constructing a source input x_s (`plainText, secKey`), (ii) executing P (`AESEncryption.encryptText`) on x_s to obtain the source output y_s (`cipherText`), (iii) constructing a follow-up input x_f that satisfies $\mathcal{R}_i$ (the source output y_s also serves as the follow-up input), (iv) executing P (`AESEncryption.decryptText`) on x_f to obtain the follow-up output y_f (`decryptedText`), and (v) verifying whether the output relation $\mathcal{R}_o$ (`assertEquals(plainText, decryptedText)`) is satisfied. Note that, following the definition by Xu et al. [67], the class `AESEncryption` is the *target program* P, and `encryptText` and `decryptText` are the *target methods* (the specific components of P exercised during MT execution). `encryptText` and `decryptText` are a pair of functionally coupled methods for formulating the MR $x = decrypt(encrypt(x))$.

MT offers several advantages: (i) *Detecting faults in "non-testable" programs.* MT is particularly valuable for validating *non-testable programs* whose expected outputs for given inputs are difficult or infeasible to specify [11, 48]. Numerous empirical studies have demonstrated the effectiveness of MT in revealing faults in such scenarios [10, 36, 39, 49, 66, 67]. (ii) *Reusable oracle across inputs.* One MR can be applied to many inputs, enabling the test suite to exercise a broader range of program behaviors and thereby improve its fault-detection capability. For example, in the AES case (Listing 1), the MR $x = decrypt(encrypt(x))$ can be applied not only to an original input such as `"Hello AES!"`, but also to corner cases such as an empty string or strings containing special characters (e.g., `plainText="~!@#$%^&*()_+"`).

Goal. This motivates our work: developing a *fully automated, domain-agnostic approach* to generate MTCs directly from code. Our approach aims to bridge this gap by leveraging functional coupling between methods as the basis for formulating MRs, and by automatically generating valid MTCs that can be applied to diverse inputs to enhance test adequacy.

3 Approach: MR-Coupler

In this section, we present MR-Coupler, an automated MTC generator based on functional coupling. Figure 1 presents an overview of MR-Coupler. Given a target method and its container class as input, MR-Coupler produces a set of MTCs. Specifically, the process consists of three phases:

[1]We incorporate both inputs and outputs in the output relation to present a more general definition of output relation that fits the MRs asserting the output relation based on inputs, such as round-trip translation [67].

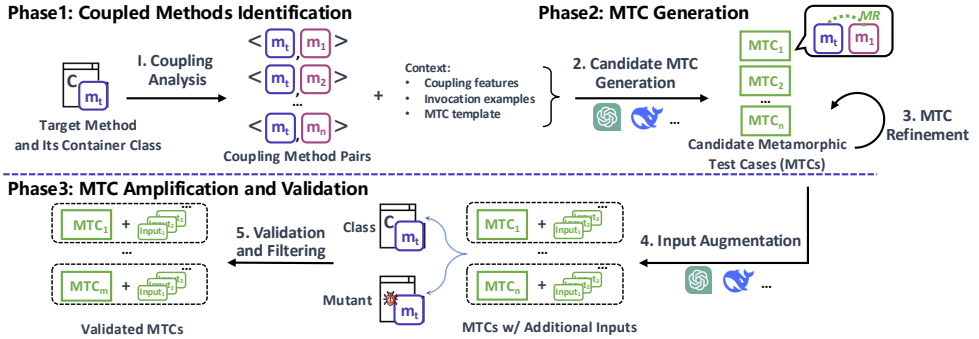


Fig. 1. An overview of MR-Coupler

(1) *Coupled Methods Identification.* In the first phase, MR-Coupler identifies functionally coupled methods that will be paired with the target method for MR construction. These methods are relevant to the target method in their intention or behavior. Such relevance can lead to potential MRs over their functionalities.
(2) *MTC Generation.* In the second phase, MR-Coupler leverages LLMs to generate MTCs for the method pairs yielded by the first phase. To mitigate the impact of LLM hallucination [77], we provide example usage of the involved methods and a template of MTC in the prompt. We also perform subsequent refinement based on the execution output of the generated MTCs.
(3) *MTC Amplification and Validation.* The third phase amplifies the MTCs generated by the second phase with additional inputs. This phase serves two goals. First, this serves as a validation for the MTC based on a necessary property: a valid MTC with additional inputs (Listing 4) should not have a lower pass rate on the original version than on the mutants. Second, the amplified MTCs can help reveal more bugs by exercising a broader range of program behaviors.

3.1 Phase 1: Coupled Methods Identification

This phase is to identify the functionally coupled methods of the target method for MR construction. However, given a target method and its container class, it is non-trivial to identify the functionally coupled methods that are suitable for MR construction.

On the one hand, classes often contain dozens of methods. For example, in our dataset, there are over 30 candidate methods in a class for each target method on average. Naively including all candidates not only increases the cost of LLM tokens but also risks of introducing noisy context, which distracts LLMs. The difficulty is to identify those methods with relevance (e.g., producer–consumer, equivalent or inverse functions) suitable for MR construction.

3.1.1 Characterization of Method Coupling. Although the number of possible methods is large, our investigation of human-written MTCs reveals that functionally coupled methods used for MR construction often exhibit coupling features in their *Intent* (as reflected by method signatures) and/or *Behavior* (as reflected by function calls and state interactions).

At the *Intent* level, method couplings are often reflected by their signature commonality (i.e., common naming tokens, parameter types, and return types). For example, the methods `encryptText` and `decryptText` in Listing 2 have complementary intent: the former encrypts a plaintext string, while the latter decrypts a ciphertext. This enables constructing an MR: $x = decryptText(encryptText(x))$. Such coupling can be mechanically inferred from their signatures, and this also aligns with prior work on semantic coupling measurement via identifiers and comments [22, 45].

Beyond the intent, coupled methods often exhibit *Behavior* coupling through their function calls and state read/write interactions. For example, the methods `insertElement` and `getElements`

Algorithm 1 Coupling Analysis for Identifying Functionally Coupled Methods

input: Target method m_t, all methods $\mathcal{M}$ in the same class C
output: Coupled method pairs $\mathcal{M}_c$ and corresponding features $\mathcal{F}$

```
1  M_c ← ∅
2  F ← empty mapping
   ▷ static features for each method
3  name(m) ← get the method name of m
4  nameTok(m) ← tokenize method name of m
5  paraRetType(m) ← get parameter and return types of m
6  calls(m) ← get invoked APIs in m
7  rField(m), wField(m) ← get accessed/updated fields in m
8  for m_i ∈ M \ {m_t} do
9      P ← ∅
       ▷ Feature1: Signature Commonality
10     if name(m_i) = name(m_t) then
11         P ← P ∪ {INTENTION : Overloading method}
12     else if nameTok(m_i) ∩ nameTok(m_t) and paraRetType(m_i) ∩ paraRetType(m_t) ≠ ∅ then
13         P ← P ∪ {INTENTION : Consuming/producing the same types of data type}
       ▷ Feature2: Function Call
14     if m_t ∈ calls(m_i) or m_i ∈ calls(m_t) then
15         P ← P ∪ {BEHAVIOR : Direct call dependency (m_i → m_t/m_t → m_i)}
16     else if calls(m_i) ∩ calls(m_t) ≠ ∅ then
17         P ← P ∪ {BEHAVIOR : Invoking the same APIs}
       ▷ Feature3: State Interaction
18     if wField(m_i) ∩ rField(m_t) ≠ ∅ or wField(m_t) ∩ rField(m_i) ≠ ∅ then
19         P ← P ∪ {STATE : Direct data dependency (m_i → m_t/m_t → m_i)}
20     else if rField(m_i) ∩ rField(m_t) ≠ ∅ or wField(m_i) ∩ wField(m_t) ≠ ∅ then
21         P ← P ∪ {STATE : Sharing the same dependent/dependency}
22     if P ≠ ∅ then
23         M_c ← M_c ∪ {⟨m_t, m_i⟩}
24         F[m_i] ← P
25 return (M_c, F)
```

interact with the same object state via the field `element`: one updates it, while the other reads it. An MR can be constructed as $r \in$ `getElements()` *just after* `insertElement`(r). Similarly, behavioral coupling may also manifest in function calls, such as one method invoking the other or both calling the same functions. Such a behavior-level coupling feature also aligns with prior work on structural coupling measurement via called functions and accessed variables [2, 22]. More examples of intent- and behavior-level coupling can be found in our artifact [57].

3.1.2 Coupling Analysis. Motivated by the above coupling features observed from human-written MTCs, in this step, MR-Coupler takes the target method m_t and all the methods within its container class C as input, and identifies its functionally coupled methods. Each is combined with the target method to form a coupling method pair. Algorithm 1 summarizes the coupling analysis procedure.

Feature1: Signature Commonality. To capture *intent-level coupling*, MR-Coupler analyzes method signatures, including method names, parameter types, and return types (Lines 3–5 in Algorithm 1). Given a target method m_t, a candidate method m_i is considered if: (i) m_t and m_i share the same method name; or (ii) m_t and m_i share common name tokens and overlap in parameter or return types (Lines 10–13). Case (i) indicates method overloading, where an equivalence MR can be constructed. For example, for the overloaded methods `base642bytes(String str)` and `base642bytes(String str, String code)`, an equivalence MR can be constructed as `base642bytes`(str) = `base642bytes`$(str, code)$ (where *code* is the default charset) — *IF* the input

string *str* is converted to bytes without explicitly specifying a charset *code*, *THEN* the resulting byte array should be identical to the one obtained by explicitly using the default charset *code* [60]. Case (ii) indicates the high likelihood that the two methods perform relevant functionalities. For instance, for methods `encryptText(String plainText)` and `decryptText(byte[] byteCipherText, SecretKey secKey)`, an inversion MR can be constructed as $x = decryptText(encryptText(x))$. The matched features (e.g., shared naming tokens or parameters) are also provided to the LLM as contextual reference for the following MTC construction.

Feature2: Function Call. As method couplings may not always be reflected in signatures, MR-Coupler further captures *behavior-level coupling* by analyzing function calls. Specifically, it extracts the set of invoked APIs for each method (Line 6 in Algorithm 1) and checks: (i) whether m_t directly invokes m_i (or vice versa), indicating call dependencies; or (ii) whether m_t and m_i share invoked APIs (Lines 14–17). When m_t directly invokes m_i (or vice versa) and shows the call dependency between them, it may indicate a functionality specialization or composition relationship. Besides, sharing common invoked APIs can indicate the commonality of their functionalities. For example, `encryptText(String plainText)` can be viewed as a specialization of `encryptTextWithAbecedarium(String plainText, String abecedarium)`, as the former implicitly uses a default `abecedarium`. Both methods invoke the same APIs (e.g., `Cipher.getInstance` in Listings 2 and 5). Accordingly, an equivalence MR can be constructed as `encryptText`(x) = `encryptTextWithAbecedarium`(x, a_{def}) (where a_{def} is the default `abecedarium`).

Feature3: State Interaction. Beyond function-call behaviors, functional coupling may also be reflected by the state write/read interactions. Therefore, MR-Coupler analyzes the fields read and written by each method (Line 7 in Algorithm 1). Given m_t, a candidate method m_i is considered if: (i) m_i writes fields that m_t later reads (or vice versa), or (ii) m_t and m_i read or write the same fields, indicating shared-state access/update.

When m_i updates fields that m_t will read (or vice versa), this indicates a state interference between them. Similarly, when they access or update the same fields, this can indicate the commonality of their data interaction. MRs can be constructed over such commonality. For example, for two methods `insertElement` and `getElements`, they interact with the same field `element` of an object: the former updates it, whereas the latter reads it. Based on this, a contamination MR: $r \in$ `getElements`$()\ just\ after$ `insertElement`(r) can be constructed.

Given a target method, MR-Coupler analyzes all the methods within its container class, and identifies the coupled methods that match any of the features. Note that a coupled method can match multiple features. The matching results are collected and then provided to LLMs as contextual reference for the following MTC construction (Lines 7–14, Listing 3).

3.2 Phase 2: MTC Generation

Given the set of coupling method pairs, this phase employs LLMs to come up with MRs and generate concrete MTCs that conduct MT. However, generating valid MTCs remains challenging even with state-of-the-art LLMs [73, 75]. Many target methods require complex object instantiations, parameter configurations, or specific environmental setups, making valid method invocation difficult. Without concrete method usage examples, LLMs are prone to hallucinations, e.g., producing code with non-existent APIs. Moreover, generated MTCs must conform to the steps of MT.

To address these challenges, MR-Coupler firstly provides LLMs with contextual guidance (e.g., method invocation examples and MTC template). After prompting the LLMs to generate MTCs, MR-Coupler further refines them based on the execution output.

3.2.1 Candidate MTC Generation. To help LLMs construct valid method invocations, MR-Coupler retrieves method invocation examples from the project under test and uses them as the guidance.

```
You are an expert in Java programming and metamorphic testing, your task is to: ...
# Code of the paired method
    ```java
        byte[] encryptText(String plainText, SecretKey secKey) { ...
        String decryptText(byte[] byteCipherText, SecretKey secKey) { ...
    ```
Coupling features on the paired methods
Intention:
    * `encryptText` and `decryptText` operate on the same set of parameters and return types,
      but with different transformations.
       * `encryptText`: (String, SecretKey) -> byte[] , `decryptText`: (byte[],SecretKey) -> String
Behavior:
    * both `encryptText` and `decryptText` invoke same APIs: `Cipher.getInstance(''AES'')` ...
Invocation examples ...
Skeleton of the container class ...
Deliverable
    ```java
        public class $testClass${ ... <MTC Template> ... }
    ```
```

Listing 3. Prompt Template for MTC Generation

```
public class AESEncryptionTest{
  @Test void MTC_input1() {
    String text = "Hello!"; var key = AESEncryption.getSecretEncryptionKey();
    byte[] encryptedText = AESEncryption.encryptText(text, key);
    String decryptedText = AESEncryption.decryptText(encryptedText, key);
    assertEquals(text, decryptedText);
  }
  @Test void MTC_input2() { String text = null; var key = null; ... }
  @Test void MTC_input3() { String text = "~!@"; var key = AESEncryption.getSecretEncryptionKey(); ... }
  @Test void MTC_input4() { String text = "_1234"; var key = AESEncryption.getSecretEncryptionKey(); ... }
  @Test void MTC_inputM() { String text = ""; var key = AESEncryption.defaultKey; ... }
}
```

Listing 4. An amplified MTC (implementing an MR $x = decryptText(encryptText(x))$) with additional inputs

Specifically, for each method pair $\langle m_t, m_i \rangle$, MR-Coupler searches for invocations of any of the two methods in the test code. MR-Coupler scans the test files (under the `/test/` directory) in the project under test and uses `JavaParser` [26] to identify test methods annotated with `@Test`. Each test is then checked whether it invokes m_t or m_i; if so, it is collected as an invocation example. To avoid excessive consumption of the LLM context window, MR-Coupler retains only the first three examples for each method and then provides them to the LLM as contextual guidance, thereby increasing the likelihood that the generated tests correctly instantiate objects and invoke methods.

Prompt Design. Listing 3 shows a simplified prompt template for MTC generation. Referring to the prompt design in recent studies [66, 68, 70], the prompt includes: (i) a system message specifying the role of LLM and its tasks, (ii) the code of the paired methods, (iii) the coupling features of the paired methods, (iv) invocation examples, (v) the skeleton of the container class, (vi) an MTC template that specifies the required deliverable. This structured prompt provides both contextual information (ii–v) and task description (i and vi), guiding the LLM to generate syntactically correct MTCs. The details of employed LLMs and their configuration can be found in Section 4.2.1. The output of this step is a set of candidate MTCs, each implemented as a standalone test class.

3.2.2 MTC Refinement. Consistent with prior observations [19, 46, 66], LLM-generated code frequently fails to execute, commonly due to errors such as "`cannot find symbol`". These errors typically arise from two sources: (i) referencing non-existent classes, APIs, or fields, or (ii) missing dependencies (e.g., absent import statements). To deal with this, MR-Coupler refines each non-compilable or non-executable MTC. First, the error message is provided back to the LLM to request an automatically revised version. If the revised MTC still fails to execute, MR-Coupler tries to fix the missing dependencies issue by statically analyzing the code using `JavaParser` to extract unresolved class names and searching for potential classes defined or imported in the project under

```java
public static byte[] encryptTextWithAbecedarium(
  String plainText,SecretKey secKey,String abecedarium) {
  Cipher aesCipher = Cipher.getInstance("AES");
  aesCipher.abecedarium = AESEncryption.abecedarium;         // bug
  // aesCipher.abecedarium = abecedarium;                    // fix
  aesCipher.init(Cipher.ENCRYPT_MODE, secKey);
  return aesCipher.doFinal(plainText);
}
```

Listing 5. Example of buggy `encryptTextWithAbecedarium`

test, and then adds the necessary import statements. Finally, this phase results in a refined set of MTCs, which are subsequently used for amplification and validation.

3.3 Phase 3: MTC Amplification and Validation

This phase validates candidate MTCs and diversifies their inputs. Specifically, MR-Coupler amplifies each MTC with additional inputs and uses mutation analysis to refute invalid ones. Specifically, mutants are created by injecting artificial faults, and a valid MTC with additional inputs (Listing 4) is expected to pass more tests on the original program than on the mutated program. In addition, the amplified MTCs can exercise a broader range of program behaviors, thereby increasing their bug-revealing capability.

3.3.1 Input Augmentation. MR-Coupler amplifies each MTC by generating additional inputs to its MR to exercise a broader range of program behaviors, thereby enhancing its bug-revealing capability. To generate these inputs, MR-Coupler employs LLMs by appending new instructions to the conversation for MTC generation and prompts the model to: (i) review the previous conversation and context, (ii) review the previously generated MTC, (iii) apply its MR to new inputs by replacing the original input with ⟨M⟩ new inputs (such as boundary values, random data, or special characters) in the form of new test cases ($M = 10$ by default), and (iv) output the new test cases within the same class following the naming convention (`MTC_input1()`, …, `MTC_inputM()`). Listing 4 shows a simplified example of an amplified MTC with 5 new inputs.

3.3.2 Validation and Filtering. Given an amplified MTC with additional inputs, MR-Coupler executes it on both the *original* version of the target program and a *mutated* version produced by injecting faults using Major [40], and the pass rates p (on the original version) and p' (on the mutated version) are recorded.

A valid MTC with additional inputs is expected to pass more tests on the original program than on the mutated program. When $p > p'$ or $p = p' = 100\%$, the MTC is retained. Considering a mutant (`encryptText` uses wrong `key` shown in Listing 2) and a valid MR $decrypt(encrypt(x)) = x$ with additional inputs in Listing 4, most inputs pass on the original version ($p = 80\%$, except when `text=NULL` throws an illegal input exception) but fail on the mutant ($p' = 20\%$, since only the case where `key = AESEncryption.defaultKey` coincidentally matches the default key succeeds). This MTC is retained. By contrast, we observed an LLM-generated invalid MR $encrypt(plainText, secKey) = encryptTextWithAbecedarium(plainText, secKey, abecedarium)$. When tested against a mutant where `encryptTextWithAbecedarium` fails to set up the `abecedarium` (Listing 5), most inputs in Listing 4 fail on the original version ($p = 20\%$, except when the user-defined `abecedarium` happens to match the default). On the mutant, all inputs pass ($p' = 100\%$) because the `abecedarium` is ignored entirely. Since $p < p'$, this MTC is filtered out.

When $p = p' = 100\%$, two interpretations are possible: (i) the injected mutants are ineffective and do not affect the tested behavior, or (ii) the MTC is ineffective in exposing mutants. In such cases, MR-Coupler conservatively retains the MTC.

After validating each generated MTC, MR-Coupler finally outputs validated MTCs.

4 Evaluation

In this section, we present our evaluation of MR-Coupler. Specifically, we aim to answer the following research questions (RQs).

RQ1 Validity: *How effective is MR-Coupler at generating MTCs?* This RQ investigates the overall effectiveness of MR-Coupler, in terms of the validity and correctness of the generated MTCs. In addition, we compared it with vanilla-LLM-based baselines to understand its superiority.

RQ2 Bug-Revealing Capability: *How effective is MR-Coupler in revealing real-world bugs?* Compared to seeded bugs (e.g., mutants), real-world bugs are often more sophisticated. Thus, we evaluate whether the MTCs generated by MR-Coupler can detect real-world bugs as the human-written MTCs do.

RQ3 Ablation Study: *How does each step contribute to the effectiveness of MR-Coupler?* MR-Coupler consists of three key steps: *Coupling Analysis*, *Input Augmentation*, and *Validation and Filtering*. This RQ performs an ablation study to assess the contribution of each step.

RQ4 Similarity: *Do the MTCs generated by MR-Coupler share the same MR skeletons as human-written ones?* This RQ evaluates whether MR-Coupler-generated MTCs align with developers' practices in selecting coupled methods and constructing input/output relations.

RQ5 Cost: *What is the cost of MR-Coupler in practice?* This RQ investigates the computational cost of MR-Coupler in terms of LLM token consumption and monetary expenditure. This is essential for assessing the practical scalability of MR-Coupler in the real world.

RQ6 Comparison Study: *How does MR-Coupler outperform MR-Scout?* MR-Scout [67] is a recent technique to synthesize MRs from MTCs. To understand how MR-Coupler compares to MR-Scout, we contrasted the two along *applicability*, *scalability*, and *correctness*.

4.1 Datasets

We prepared two datasets to answer the above RQs. The first dataset includes pairs of target methods together with corresponding human-written MTCs available in open-source projects to evaluate the validity and similarity of the generated MTCs (RQ1 and RQ4). The other is a subset from the first dataset, including only the cases whose MTCs can reveal bugs on a historical buggy version of the target program, for evaluating the bug-revealing capability of generated MTCs (RQ2).

4.1.1 Human-Written MTCs. The first dataset contains 1,471 MTCs written by developers in open-source Java projects. Each entry in this dataset consists of a human-written MTC and a corresponding pair of MR-coupled methods. These MTCs are valid and executable. Such a dataset is leveraged to (i) evaluate the validity of automatically generated MTCs (RQ1), by running them on executable target methods, and (ii) measure the similarity between the human-written MTCs and MR-Coupler generated MTCs, by checking whether they encode the same MR-skeletons (RQ4).

To construct such a dataset, we adopted a strategy similar to Xu et al. [67]. Specifically, we collected a list of high-quality Java projects (i.e., with at least 50 stars) from GitHub. The query was done on December-16, 2024, which returned over 24,000 projects. We then ran MR-Scout [67] to discover human-written MTCs from these projects, which yielded 46,006 candidate MTCs. With these candidates, we applied three filtering criteria to select the valid and executable MTCs: (i) they must compile, as we need to compile and run the test cases in our experiments; (ii) they must pass in the latest version of the project to ensure the MTCs are valid; and (iii) the commit introducing these MTCs must mention an issue number in its commit message (e.g., containing "`#123`"). We prioritize such tests since they are often extensively discussed and reviewed to disclose the issues and thus tend to be of high quality.

The whole process yielded a dataset containing 1,471 entries. Each entry in this dataset consists of a human-written MTC and a corresponding pair of MR-coupled methods. The 1,471 MTCs

come from 213 Java projects, which on average contain 3,505.44 commits, 1,772.73 Java files, and 199,602.38 lines of Java code. Detailed information about these MTCs and projects is available on the MR-Coupler website [57]. We ran MR-Scout [67] to obtain the corresponding pair of MR-coupled methods for each MTC. For example, in the MTC that encodes the relation $x = decrypt(encrypt(x))$ (Listing 1), `encryptText` and `decryptText` are the MR-coupled methods and will be identified by MR-Scout. We use the first invoked method `encryptText` as the target method and take `decryptText` as the ground truth of a coupled method. Each entry formulates an MTC generation task used for our experiments of RQ1 and RQ4.

4.1.2 Bug-revealing MTCs. The other dataset is made up of 50 entities with bug-revealing MTCs filtered from the first dataset. These entities are used to evaluate whether MR-Coupler can generate effective MTCs to reveal real bugs as the human-written MTCs do (RQ2). We identify such entities from the first dataset by checking whether their MTC fails on a buggy version and passes on a fixed version. Specifically, an issue report may be resolved through commits; thus, for each issue-associated MTC, we identify two versions of the project: (i) the potential *buggy version*, defined as the commit before all issue-related commits; (ii) the potential *fixed version*, defined as the last commit of all issue-related commits. We then execute the MTC on both versions. We consider an MTC bug-revealing only if it fails on the buggy version and passes on the fixed version.

This process required significant manual effort to set up specific project environments for multiple versions of each project and resolve complicated dependency issues. We ultimately reproduced 50 MTC-bug pairs, obtaining their corresponding buggy and fixed program versions, which form the benchmark for evaluating the bug-revealing capability of MR-Coupler (RQ2).

4.2 Evaluation Setup

This section introduces the employed LLMs, baselines, and the experiment environment.

4.2.1 Employed Large Language Models. MR-Coupler employs LLMs to generate MTCs and their alternative inputs. In the evaluation, we include representative state-of-the-art LLMs [20], covering general-purpose, coding, and reasoning LLMs from well-known model families. Specifically, they are *GPT-4o mini* from OpenAI [42],*Qwen3-coder-Flash* from Alibaba [1], *DeepSeek-V3.1* and *DeepSeek-V3.1-Think* from DeepSeek [16]. Following a typical setup in recent studies [9, 18, 66], for each MTC generation task, we repeated the generation process five times with a temperature setting of 0.2.

4.2.2 Baselines. To the best of our knowledge, there is no existing fully automated and domain-agnostic approach to generate metamorphic test cases for a given program under test. Although some approaches are proposed to generate domain-specific MRs [71, 74], or synthesize MRs based on human-prepared materials [41, 66, 67] or manual effort [50] (discussed in Section 6), adapting them into comparable automated domain-agnostic baselines is non-trivial. Given the proven effectiveness of LLMs in code [32, 35, 66] and test generation [53, 70], we set *directly prompting LLMs* as a baseline. In this baseline, we allow LLMs to conduct a round of revision to the generated code based on the execution feedback as in our method, which is found to be an effective common post-processing to enhance code generation [70]. The baseline uses a similar prompt template (Listing 3), and follows the same refinement step in Section 3.2.2. Experimental results show that a target method can be paired with 6.93 relevant methods (rounded to 7) by MR-Coupler on average. Therefore, to have a fair comparison, we instructed the baseline LLMs to generate 8 candidate MTCs, which correspond to the target method itself, plus the 7 additional relevant methods. For each task, we repeat the generation process five times (temperature: 0.2), consistent with MR-Coupler's configuration.

Table 1. Effectiveness of MR-Coupler in Generating Valid MTCs for 100 Target Methods

Metric	GPT-4o-mini		Qwen3-coder-Flash		Deepseek-V3.1		Deepseek-V3.1-Think		**Improv.**
	Baseline	MR-Coupler	Baseline	MR-Coupler	Baseline	MR-Coupler	Baseline	MR-Coupler	
Num. of Generated TCs	3923	4176	3984	3911	3968	3626	3971	4151	-
Pct. of Executable MTC	50.40%	83.69%	60.02%	92.66%	60.26%	93.08%	62.08%	88.44%	**54.35%**
Pct. of Valid MTC	40.66%	71.84%	47.52%	80.98%	52.60%	84.94%	55.35%	83.57%	**64.90%**
Num. of Successful Tasks	62	92	76	91	82	95	85	98	**24.82%**
Pct. of False Alarm	19.32%	14.16%	20.70%	12.61%	12.71%	8.74%	10.83%	5.50%	**36.56%**

4.2.3 Experimental Environment. All experiments were conducted on a machine with a 64-core AMD Ryzen Threadripper PRO 3995WX CPU and 512 GB RAM. The LLMs in our evaluation are running on cloud platforms and accessed via the official APIs of OpenAI, Alibaba, and DeepSeek.

4.3 RQ1: Validity of Generated MTCs

RQ1 aims to evaluate the overall effectiveness of MR-Coupler in generating valid MTCs. To this end, we run MR-Coupler on the target methods in the dataset of human-written MTCs. We also compare it against the baselines (Section 4.2).

4.3.1 Experiment Setup. Experiments were conducted at scale using four LLMs, with each MTC generation task repeated five times (Section 4.2.1). Running MR-Coupler and the baselines on all 1,471 tasks is time-consuming and unaffordable. Therefore, we evaluated MR-Coupler and baselines on 100 randomly sampled entries. This sample size ensures a 95% confidence level with a margin of error $\leq 10\%$. The randomly sampled 100 entries come from 55 projects. The details are available on MR-Coupler's site [57]. For each entry (i.e., target method), MR-Coupler generates multiple MTCs. We present the total number of generated test cases, and measure the effectiveness using the following metrics:

- *Percentage of Executable MTC:* The proportion of generated test cases that compile, run without error, and satisfy the two necessary properties of an MTC [67]: (i) at least two invocations of target methods; (ii) one assertion relating those invocations' inputs and outputs. Xu et al. [67] reported, MR-Scout can achieve 97% precision, and we ran MR-Scout on each case.
- *Percentage of Valid MTC:* The proportion of valid MTCs to all generated test cases, where a valid MTC is an executable MTC that passes on the latest project version. We assume the latest version of a target method is of low probability to be buggy, as it has passed human-written MTCs.
- *Number of Successful Tasks:* The number of target methods generated with at least one valid MTC.
- *Percentage of False Alarm:* The proportion of invalid MTCs to all executable MTCs, where an invalid MTC satisfies the properties but fails on the latest version.

4.3.2 Experiment Results. Table 1 shows the results of MR-Coupler in generating valid MTCs for 100 target methods. MR-Coupler generated valid MTCs for over 90 target methods, achieving its best performance with *DeepSeek-V3.1-Think*. MR-Coupler produced valid MTCs for 98 of 100 target methods with a 5.5% false alarm rate. Compared to baseline, MR-Coupler improves the valid MTC rate by 64.90% and reduces false alarms by 36.56%. Even with the weakest model (*GPT-4o mini*), MR-Coupler outperforms baselines using stronger models (e.g., *DeepSeek-V3.1-Think*).

The improvements are from two key factors. On the one hand, providing LLMs with functionally coupled methods serves as a hint, effectively inspiring them to infer valid MRs and then generate valid MTCs, thereby reducing hallucinations. Without such context, coming up with MRs from scratch is challenging for LLMs, leading to higher rates of invalid MRs. On average, MR-Coupler identified 6.93 relevant methods per target method from 30.44 candidate methods in their container classes, significantly narrowing the enumeration space. On the other hand, retrieving real invocation

examples helps LLMs construct valid input objects and correctly invoke methods, particularly when methods require complex object instantiations. This context helps generate 83.69% to 93.08% executable MTCs, which are 54.35% more compared with baselines.

Failure analysis. When built with *DeepSeek-V3.1-Think*, MR-Coupler fails to generate any valid MTCs for two target methods. This is because, for one target method, execution requires access to external or environmental resources (e.g., a JSON file or environment variable), but no invocation examples were available in the repository as the context. As a result, MR-Coupler failed to configure these resources in the generated MTCs, leading to non-executable tests and no valid MTCs. For the other target method, MR-Coupler generated all executable but invalid MTCs (i.e., false alarms).

MR-Coupler exhibits a 5.5% false-alarm rate when built with *DeepSeek-V3.1-Think*. There are two main reasons. (i) MR-Coupler relies on Major [40] to generate mutants for validation (Section 3.3.2). For some target methods, Major failed to execute due to environmental issues (e.g., uncompilable dependencies), preventing mutant generation and disabling the validation step that filters false alarms. (ii) For some cases, the generated MTCs cannot reveal the injected mutants, and do not expose behavioral differences between the base and mutated versions – the pass rates are identical, causing MR-Coupler to retain false alarms.

4.3.3 Correctness of Generated MTCs. While the above automatic and objective metrics evaluate the validity of generated MTCs by checking the necessary properties and their execution results, this does not guarantee the semantic correctness of the underlying MRs. A generated MTC may satisfy these validity criteria yet still be semantically incorrect or meaningless (e.g., vacuous oracles). To assess the semantic correctness, we further conduct a manual study to validate whether generated MTCs indeed reflect correct MRs for the target method.

Setup. For each of the 100 target methods in RQ1, we manually inspected one randomly selected MR-Coupler-generated MTC. Two participants (each with over four years of experience in Java and metamorphic testing) independently validated each selected MTC. For cases with disagreement, the two participants discussed and reached a consensus.

An MTC is deemed *correct* if it reflects an MR present in the code; it may miss failures but must not produce false alarms. For each MTC, participants first understand the functionality of the target method and its coupled method. They then analyzed the underlying MR an MTC and determined whether an MTC is correct. Particularly, they assess whether any valid input could violate the assertion and checked execution for false alarms. If any of the cases is found, the MTC is incorrect.

Result. Among the 100 sampled MTCs, 8 were *Incorrect*, and 3 are too domain-specific and complicated to understand and validate their correctness. The incorrect cases mainly involved faulty test implementations (e.g., triggering `IllegalArgumentException` or `NullPointerException`) or MRs not holding for corner-case inputs. The remaining 89 MTCs were *Correct*. The inter-rater agreement between participants was high (87%), and disagreements were resolved through discussion. Overall, these results suggest MR-Coupler's effectiveness in generating correct MTCs.

Answer to RQ1: MR-Coupler successfully generates valid MTCs for over 90% of tasks, achieving 64.90% and 36.56% improvements in generating valid MTCs and reducing false alarms, respectively, compared with baselines. Our manual validation shows that only 8 out of 100 sampled generated MTCs were incorrect.

4.4 RQ2: Bug-revealing capability

4.4.1 Experiment Setup. This RQ evaluates the capability of MR-Coupler in revealing real bugs, especially for those originally discovered by metamorphic test cases. With the collected 50 MTC-bug

Table 2. Bug-Revealing Results of MR-Coupler on 50 Bugs

Metric	GPT-4o-mini		Qwen3-coder-Flash		Deepseek-V3.1		Deepseek-V3.1-Think		Improv.
	Baseline	MR-Coupler	Baseline	MR-Coupler	Baseline	MR-Coupler	Baseline	MR-Coupler	
Num. of Generated TCs	1987	2740	1976	2086	2024	1797	2007	2661	-
Pct. of Bug-revealing MTCs	3.84%	6.53%	4.14%	7.29%	5.21%	6.60%	3.92%	7.77%	**67.65%**
Num. of Revealed Bugs	4	15	5	20	7	16	7	22	**229.46%**

pairs (Section 4.1), for each bug, we take the buggy method as the target method, and measure the bug-revealing capability by the following two metrics:

- *Percentage of Bug-Revealing MTCs:* The proportion of generated MTCs that are bug-revealing, where a bug-revealing MTC is defined as an executable MTC that fails on the buggy version but passes on the fixed version of the target method.
- *Number of Revealed Bugs:* The number of bugs for which at least one generated MTC fails on the buggy version and passes on the fixed version.

4.4.2 Experiment Result. As shown in Table 2, MR-Coupler (*DeepSeek-V3.1-Think*) performed the best, successfully revealing 22 real-world bugs. Compared with the baseline, it uncovers 15 additional bugs and improves the bug-revealing rate by 98.21%. When combined with other models, MR-Coupler consistently outperforms baselines, revealing 9∼15 additional bugs with an average 67.65% increase in bug-revealing MTC rate.

Based on manual inspection, we attribute the improvement to two main factors: (i) *Coupling-aware MR construction.* Some bugs are exposed only by MRs over specific method pairs, and MR-Coupler can generate such MRs through its effective coupled methods identification. For example, a bug [27] in the method `randomRepo` is revealed when coupled with the method `repos`, as both invoke `MkRepo` and access shared fields (`storage`, `self`). This allows MR-Coupler to identify the coupling and then generate an effective MR. (ii) *Input augmentation.* Some bugs require specific input to trigger. By applying additional inputs (Section 3.3.1), MR-Coupler exercises diverse behaviors and exposes corner cases, such as boundary inputs in `previousClearBit` (e.g., `i = 1«16`) [62]. In addition, among all evaluated LLMs, *DeepSeek-V3.1-Think* reveals the most bugs, likely due to its reasoning ability. By analyzing the code and understanding the coupling between methods, it can identify fault-prone interactions and construct effective MRs.

By analyzing the overlap of bugs revealed by MR-Coupler with different LLMs, we observed that a total of 28 bugs were revealed, with 8 detected by all models and both *DeepSeek-V3.1-Think* and *Qwen3-coder-Flash* uniquely revealed two additional bugs. This suggests that *combining models could further improve the bug-revealing capability.* This is an interesting strategy for future enhancement. Notably, 19 of the 22 bugs found by *DeepSeek-V3.1-Think* are exposed by multiple distinct MRs. For example, a bug in `multiply` can be revealed by different algebraic relations [43]. This highlights that *MR diversity plays a role in enhancing the bug-revealing capability.*

Failure analysis. While MR-Coupler successfully detected 22 (44%) real bugs originally revealed by human-written MTCs, some bugs remained unrevealed. A major reason is that some buggy methods are highly domain-specific and implement complex business logic, where providing only the method code is insufficient for LLMs to infer the intended behavior (e.g., a bug related to "compaction file metrics" in Apache IoTDB [25]). Module-level or project-level knowledge is required. Automatically extracting and providing such knowledge as the context for LLMs remains a challenging and promising direction for future research. In other cases, constructing inputs required access to external resources, such as environment variables or specific file contents [79]. When no concrete examples were retrieved in the project under test, MR-Coupler-generated MTCs failed to set up that, resulting in non-executable or non-bug-revealing tests. Effectively retrieving and adapting such project-level context for test generation is still an open challenge.

Table 3. Ablation Study on MR-Coupler (*DeepSeek-V3.1-Think*)

Metric	MR-Coupler	v_1: w/o Coupling Analysis	v_2: w/o MTC Amplification	v_3: w/o MTC Validation
Num. of Generated TC	4151	3367	4209	4367
Pct. of Executable MTC	88.44%	63.86% (-27.80%)	88.69% (0.29%)	88.99% (0.64%)
Pct. of Valid MTC	83.57%	**56.28% (-32.65%)**	83.61% (0.05%)	78.84% (-5.66%)
Num. of Successful Task	98	**86 (-12.24%)**	98 (0.00%)	98 (0.00%)
Pct. of False Alarm	5.50%	**11.86 (115.53%)**	5.73% (4.13%)	**9.44% (71.56%)**
Pct. of Bug-revealing MTC	7.77%	**3.37% (-56.62%)**	**3.89% (-49.92%)**	14.06% (81.04%)
Num. of Revealed Bugs	22	**12 (-45.45%)**	**13 (-40.91%)**	24 (9.09%)

Answer to RQ2: MR-Coupler can successfully detect 22 (out of 50) real-world bugs originally discovered by human-written MTCs. However, some unrevealed bugs are rooted in domain-specific business logic, requiring module-level or even project-level context to construct bug-revealing MTCs. Effectively leveraging such context remains an open challenge for future work.

4.5 RQ3: Ablation Study on MR-Coupler

4.5.1 Experiment Setup. This RQ aims to evaluate the contribution of major steps in MR-Coupler to its overall effectiveness in generating valid and bug-revealing MTCs. We use the same tasks and metrics as in RQ1 (validity) and RQ2 (bug-revealing capability). We created three ablated variants of MR-Coupler (v_1, v_2, and v_3) by ablating three steps to analyze their contribution. We chose MR-Coupler built with *DeepSeek-V3.1-Think* which achieves the best result in RQ1 and RQ2 (Sections 4.3 and 4.4). The variants are as follows:

- v_1**: MR-Coupler w/o Coupling Analysis.** This variant disables the *Coupling Analysis* step (Section 3.1), meaning no functionally coupled methods and corresponding invocation examples are provided as context to LLMs during the MTC generation.
- v_2**: MR-Coupler w/o MTC Amplification.** This variant disables the *Input Augmentation* step (Section 3.3.1), thus no additional inputs are generated to amplify MTCs.
- v_3**: MR-Coupler w/o MTC Validation.** This variant disables the *Validation and Filtering* step (Section 3.3.2), meaning all generated MTCs are retained without filtering.

4.5.2 Experiment Result. As shown in Table 3, disabling *Coupling Analysis* (v_1) led to a 32.65% decrease in valid MTC rate and a 56.62% decrease in bug-revealing rate. This suggests that leveraging functional coupling as an explicit hint is crucial for generating valid MTCs and reducing hallucinations. This aligns with the findings in RQs of validation and bug-revealing capability.

When disabling *Input Augmentation* (v_2), the number of revealed bugs significantly decreased by 40.91% (from 22 to 13). This highlights that applying generated MRs to additional inputs strengthens generated MTCs by exercising a wider range of program behaviors. Nevertheless, even without input augmentation, MR-Coupler revealed more bugs compared with the baseline, indicating that MRs over multiple methods already contribute to bug revealing, and MTC amplification further boosts the bug-revealing rate by 89.94% (from 3.89% to 7.77%).

It is observed that when disabling *Input Augmentation*, MR-Coupler still successfully finished 98 tasks (i.e., generating valid MTCs for 98 target methods). This is because *Input Augmentation* is designed to apply generated MRs to more test inputs, rather than assist in generating valid MRs. These 98 tasks were already generated with valid MTCs prior to *Input Augmentation*, which then augments them with more diverse inputs. However, without *Input Augmentation*, validation based on mutation analysis becomes less effective due to reduced input diversity. As a result, more MTCs are retained (from 4,151 to 4,209), and the false-alarm rate slightly increases (from 5.50% to 5.73%).

Disabling *Validation and Filtering* (v_3) increased the false-alarm percentage by 71.56% (from 5.5% to 9.44%), indicating the effectiveness of the validation step in mitigating the false alarm issue. The

Table 4. Similarity of MR-Coupler-Generated MTCs to Human-Written MTCs

Metric	GPT-4o-mini		Qwen3-coder-Flash		Deepseek-V3.1		Deepseek-V3.1-Think		**Improv.**
	Baseline	MR-Coupler	Baseline	MR-Coupler	Baseline	MR-Coupler	Baseline	MR-Coupler	
L1: Method-Pair Consistency	61%	89% (+45.90%)	61%	86% (+40.98%)	74%	87% (+17.56%)	81%	92% (+13.58%)	**29.51%**
L2: MR-Skeleton Consistency	46%	85% (+84.78%)	46%	84% (+82.61%)	55%	84% (+52.73%)	65%	90% (+38.46%)	**64.65%**

slight increase in bug-revealing rate is because some bug-revealing MTCs are filtered out together with invalid ones. In some cases, both invalid and bug-revealing MTCs fail on both the original and mutated versions (0% pass rate), or valid MTCs fail to kill any mutants (100% pass rate on both), making mutation analysis based validation unable to distinguish them. A possible mitigation is to generate more diverse inputs to improve the mutant-killing capability.

Answer to RQ3: Each of the three steps uniquely enhances MR-Coupler's effectiveness. Functional coupling helps LLMs to generate more valid MTCs, MTC amplification augments the input to reveal more bugs, and mutation analysis based validation filters nearly half of the false alarms (reducing the rate from 9.44% to 5.5%).

4.6 RQ4: Similarity to human-written MTCs

4.6.1 Experiment Setup. Human-written MTCs represent well-established practices for constructing MRs, including the selection of method pairs as well as the input and output relation construction. This RQ evaluates whether the MTCs generated by MR-Coupler can mimic these practices by checking if they encode the same *MR-skeletons* as human-written ones. This can demonstrate the potential of MR-Coupler to assist developers in MTCs construction, facilitating developers in integrating the generated tests into their codebase and easing subsequent maintenance. We use the same 100 evaluation tasks as in RQ1.

According to the definition of MTC in Section 2, an MR-skeleton consists of three core components: (i) *Input Relation:* the input transformation (e.g., API calls) applied to generate follow-up inputs, if applicable; (ii) *Execution:* the MR involved method pair (e.g., `<encryptText, decryptText>` in Listing 1); and (iii) *Output Relation:* the assertion type (e.g., `assertEquals`) and the involved elements (e.g., source input, source output, follow-up output) to verify the output relation. Considering that the same output relation can be implemented in multiple ways, we normalize assertions for ease of comparison. Specifically, we normalize all assertions into comparable assertions [67]. For example, boolean-style `assertTrue(x.equals(y))` and `assertFalse(x.equals(y))` are normalized to `assertEquals(x, y)` and `assertNotEquals(x, y)`, respectively. More details can be found in MR-Coupler's artifact [57]. Based on the definition of MR-skeleton, we take the human-written MTCs as the ground truth, and measure the similarity at two levels:

- L1: Method-Pair consistency: the proportion of target methods where at least one generated MTC couples the *same method pair* as the human-written MTC.
- L2: MR-Skeleton consistency: the proportion of target methods where at least one generated MTC encodes the same MR-skeleton as the human-written MTC, i.e., matching the input transformation, method pair, and output relation assertion type and elements. The MTCs satisfying L2 must satisfy L1 as well.

4.6.2 Experiment Result. Table 4 shows that MR-Coupler-generated MTCs can match the human-coupled method pairs for 86~92 target methods and encode the same MR-skeletons for 84~90. Compared to the baseline, MR-Coupler improves method-pair consistency by 29.51% and full MR-skeleton consistency by 64.65%. These results highlight the effectiveness of MR-Coupler's

coupling analysis. MR-Coupler identified most functionally coupled methods used in human-written MTCs. The high MR-skeleton consistency further demonstrates MR-Coupler's potential to assist developers in MTCs construction, integration, and maintenance.

Failure analysis. Despite the overall high consistency, MR-Coupler (*DeepSeek-V3.1-Think*) missed eight tasks in identifying the same method pairs and failed to encode the same MR-skeleton in ten tasks. Our inspection revealed three main causes: (i) some developer-selected method pairs exhibit implicit relevance in the code, such as `a2q` paired with `readAndWrite` for JSON serialization [21]; (ii) MR-Coupler-generated MTCs sometimes construct correct but different MRs, e.g., for the `cosineSimilarity` method, the developer-constructed MR captures self-similarity dominance ($\cos(x, x) \geq \cos(x, y)$ for any vector $y \neq x$), while MR-Coupler derives an equivalence MR ($\cos(x, y) = \cos(x, y)$) [17]; and (iii) some inconsistencies arise from equivalent but differently expressed assertions, e.g., `assertEquals(x,y)` and `assertTrue(x.customizedEquals(y))` [37]. The concrete MTCs can be found at MR-Coupler's site [57].

Answer to RQ4: MR-Coupler can identify most of the coupled method pairs used in human-written MTCs, achieving over 90% MR-skeleton consistency with human-written MTCs. This demonstrates its potential to assist developers in MTC construction.

4.7 RQ5: Cost Analysis

To better understand the practical scalability of MR-Coupler, we analyze the cost of using LLMs in terms of token consumption and monetary expenditure. We report the average cost of generating MTCs for a single target method, aggregated over all LLM invocations involved in MR-Coupler, including five repeated runs of MTC generation (Section 4.2), as well as MTC Amplification and Validation steps in MR-Coupler. On average, for one target method, MR-Coupler costs \$0.09 (527.96k) using GPT-4o-mini, \$0.23 (501.72k) using Qwen3-coder-Flash, \$0.10 (533.02k) using Deepseek-V3.1, and \$0.40 (736.35k) using Deepseek-V3.1-Think. Deepseek-V3.1-Think consumes 38.24% more tokens than Deepseek-V3.1, which we attribute to additional reasoning tokens during inference. Moreover, Deepseek-V3.1-Think has a higher token price than Deepseek-V3.1, resulting in the highest cost (\$0.40 per target method). Note that our experiments were conducted in August 2025; token pricing and discount policies may change over time.

Answer to RQ5: MR-Coupler consumes around 501k~736k tokens (\$0.09~\$0.40) per target method averagely. The cost is comparable to recent LLM-based test generation approaches [3, 13].

4.8 RQ6: MR-Coupler V.S. MR-Scout

To better clarify the value of MR-Coupler, we conducted a direct comparison between MR-Coupler and MR-Scout [67] (a recent technique that discovers and synthesizes MRs from existing developer-written MTCs) from three perspectives: *applicability*, *correctness*, and *scalability*.

4.8.1 Applicability. MR-Scout suffers from two fundamental limitations in applicability. First, it synthesized MRs from existing MTCs, and thus only works for programs (i.e., methods under test) that already have developer-written MTCs. However, MTCs are rare (1%) in real-world projects [67], which limits its applicability. In contrast, MR-Coupler infers MRs directly from the target program. It complements MR-Scout by steering away from the reliance on existing MTCs.

Second, MR-Scout requires explicit input transformations in the discovered MTCs. If such transformations are missing, MR-Scout cannot synthesize codified MRs from the discovered MTCs, and therefore, cannot generalize them to new inputs as new MTCs. For the 100 target methods with existing MTCs in RQ1, MR-Scout is only applicable to 21 of them, due to the absence of such

Table 5. Performance of MR-Coupler (GPT-4o-mini) on 100 Target Methods Before or After the Cut-Off Date

Target methods	Validity			Similarity	
	Num. of Successful Tasks	Pct. of Valid MTCs	Pct. of False alarm	L1: Method-Pair consistency	L2: MR-Skeleton consistency
Before Cut-off	92	71.84%	14.16%	89	85
After Cut-off	94	73.17%	9.52%	85	79

transformations in the others. In comparison, MR-Coupler successfully generates valid MTCs amplified with diverse inputs for 98 of the 100 target methods (Section 4.3).

4.8.2 Correctness. For the 100 target methods in RQ1, our manual validation of the 100 sampled MTCs shows that MR-Coupler achieves a correctness rate of 0.89 (Section 4.3.3). For MR-Scout synthesized MRs, we manually validated them using the same protocol in Section 4.3.3, and the result shows a correctness rate of 0.90. Overall, these results suggest that MR-Coupler achieves comparable correctness to MR-Scout.

Differences and commonalities. We further analyze the overlap between MTCs constructed by MR-Coupler and MR-Scout. For only 14 out of the 100 target methods, MR-Coupler and MR-Scout generated MTCs encoding the same MR, indicating that the overlap is minor and thus the two approaches derive complementary MRs in practice.

4.8.3 Scalability. We compare scalability in terms of LLM cost and end-to-end runtime. MR-Scout does not rely on LLMs and thus incurs no token or monetary cost. MR-Coupler introduces additional LLM overhead, consuming 501k~736k tokens ($0.09~$0.40) per target method. MR-Coupler takes 3.34 minutes per target method on average (using Deepseek-V3.1-Think). MR-Scout's runtime depends on the EvoSuite time budget (20 minutes by default) for input generation during MR validation, and its MR discovery and synthesis phases take another 20 seconds or so.

Answer to RQ6: MR-Coupler and MR-Scout are complementary, as they can derive MRs under different scenarios and can be used in conjunction. MR-Coupler largely complements MR-Scout for the scenario that no existing MTCs are available. While MR-Coupler incurs additional LLM usage costs, it targets scenarios beyond MR-Scout's applicability and achieves comparable correctness. MR-Coupler enables wider adoption of metamorphic testing.

4.9 Threats to Validity

Representativeness of LLMs. Since MR-Coupler relies on LLMs for MTC and input generation, one potential threat is whether our findings based on the selected LLMs are representative. To mitigate this threat, according to the EvalPlus leaderboard [20], we include representative LLMs from three well-known LLM families, i.e., *GPT-4o mini* from OpenAI [42],*Qwen3-coder-Flash* from Alibaba [1], *DeepSeek-V3.1* and *DeepSeek-V3.1-Think* from DeepSeek [16].

Data Contamination. A potential threat is data contamination, where target programs or MTCs may have appeared in the LLMs' pretraining data, potentially biasing results. To mitigate this, we constructed an *after cut-off* dataset following the same procedure in Section 4.1, using entries created after the GPT-4o-mini training cut-off (October 2023 [29]). As shown in Table 5, MR-Coupler achieved slightly lower similarity to human-written MTCs but a higher percentage of valid MTC compared to the *before cut-off* dataset. This indicates that MR-Coupler's effectiveness still holds for subjects after the cut-off date, and not simply an artifact of training-data memorization.

Another potential data leakage arises from providing underlying MTCs as examples during the MTC generation (Section 3.2). In the evaluation, we excluded all detected MTCs from the examples and manually inspected results for 50 buggy subjects (Section 4.1.2), finding no such leakage.

Representativeness of Experimental Subjects. A potential threat concerns the generalizability of our findings across projects and programming languages. To mitigate the first one, we selected representative Java projects (Section 4.1), following the strategy of prior studies [61, 66, 67]. Due to the substantial effort required to collect reproducible MT-revealed bugs, this study focuses on a single widely used language (Java) and evaluates the bug-revealing capability of MR-Coupler (RQ2) on 50 Java bugs. This constitutes a limitation of our study — the generalizability of our findings to other programming languages is not evaluated. Constructing a benchmark of multiple programming languages is a meaningful direction for future work.

Quality of Ground Truths. We use human-written MTCs as ground truth and treat the fixed or latest program versions as bug-free when answering the RQs on validity, bug revealing, and similarity. A potential threat concerns the quality of these ground truths. To mitigate this, we applied three filtering criteria to select high-quality MTCs and corresponding target methods (Section 4.1). We further manually validated the developer-written MTCs for the 100 sampled entries in RQ1, following the criteria of MR-Scout [67]. We found that 99 out of 100 MTCs are valid, with only one too complex to understand and validate. This aligns with the 0.97 precision reported by MR-Scout [67], indicating the high quality of these ground truths.

5 Discussion

Type of bugs that MR-Coupler can detect. In the evaluation, the experimental bugs are known to be detectable by MTCs (Section 4.4). However, MR-Coupler is not limited to bugs that are necessarily found with MTCs. This is because MTCs generated by MR-Coupler do not target specific types of bugs. For example, in our experiments, Bug #49 [44] arises from integer overflow in `multiply`, causing an incorrect sign flip when the numerator or denominator is large. Given a large numerator or denominator, this bug can be revealed by a non-MR assertion (e.g., `assertTrue(a.multiply(b)>0)`), or an MR-Coupler-generated MTC over the coupled methods `multiply` and `divide` — `assertEquals(a.multiply(b)/b, a.divide(b)*b)`.

Capability boundary of MR-Coupler. MR-Coupler is designed to leverage intra-class coupling for MR generation. Specifically, it identifies functionally coupled method pairs within the same class as ingredients for MR construction. MRs that cannot be captured by these coupling features are therefore not generated, and MR-Coupler currently does not support inter-class coupling. Generating MRs over inter-class coupling presents unique challenges, such as a substantially larger search space of candidate methods and classes, and is beyond the scope of this paper. We consider it an interesting direction for future work.

6 Related Work

LLM-Based Automated Test Generation. Recent work has demonstrated the broad utility of LLMs in software testing [55], including unit test generation [47], fuzzing [28], and test-oracle construction [7]. Lemieux et al. [31], Tang et al. [53], Yuan et al. [70] studied LLM-based unit test generation and reported substantial potential as well as challenges in reliability and coverage. Xia et al. [63] further showed that LLM prompting can drive effective, language-agnostic fuzzing. Hossain et al. [24] investigated neural test-oracle generation and highlighted the need for robust validation. These advances motivate LLM-based MT, where oracle construction remains a central bottleneck.

LLM-Based MR Generation. Several recent studies have explored using LLMs to generate MRs and other MT artifacts. Xu et al. [66] proposed an LLM-based technique to automatically deduce input transformations from pairs of source and follow-up inputs. Shin et al. [50] derived MRs from requirement specifications and translated them into SMRL, a domain-specific MR language, via a two-phase LLM pipeline. Zhang et al. [74] proposed a human-AI hybrid MT framework that uses predefined MR patterns to generate MRs for autonomous driving systems. Tsigkanos et al. [59]

leveraged LLMs to discover input and output variables from scientific software documentation for constructing MRs. Hazott and Große [23] designed a multi-step prompt engineering pipeline to generate MRs for embedded graphics libraries, nearly doubling structural coverage and uncovering 14 new bugs compared to manually crafted MRs. These approaches show promise but either target specific domains, depend on auxiliary artifacts (e.g., requirement documents or API manuals), or require substantial human intervention.

Some studies evaluated LLM capabilities for MR generation. Zhang et al. [75] conducted a pilot study using ChatGPT (3.5) for MR generation in autonomous driving and found it effective at proposing useful MR candidates. Luu et al. [38] examined ChatGPT (GPT-4) across nine systems of varying complexity and reported that, while correct MRs can be generated, the majority of candidates were either vaguely defined or incorrect, especially for complex or previously untested systems. Zhang et al. [73] evaluated MR discovery across 37 subjects and found that 4.6~38.6% of existing MRs were rediscovered but only 29.9~43.8% of generated MRs were valid. Zhang et al. [76] further showed that stronger models (e.g., GPT-4) improve MR quality over weaker ones, though human expertise remains essential for adjudication and refinement. In contrast, MR-Coupler leverages LLMs' reasoning capability to derive MRs from functionally coupled methods and generate concrete executable MTCs, providing an end-to-end automated and self-validating pipeline.

Traditional MR Generation Approaches. Prior to LLMs, MR generation techniques relied on search-based, pattern-based, genetic-programming-based, or heuristic approaches. Ayerdi et al. [4, 5] and Terragni et al. [54] generated MRs via genetic programming but assumed a regression testing scenario. Zhang et al. [71] and Zhang et al. [72] proposed search-based approaches to inferring MRs for numeric programs. Zhou et al. [78] and Segura et al. [49] identified MRs based on predefined patterns, limiting generalizability across domains. Nolasco et al. [41] proposed MeMoRIA to identify equivalence MRs from documentation. Recently, Xu et al. [67] derived MRs from existing test cases. However, these approaches rely on scarce resources (i.e., MR-embedded documents or tests). In contrast, MR-Coupler directly generates MTCs from the target program without relying on such resources, and is not restricted to the regression testing scenario.

7 Conclusion

This paper presents MR-Coupler, a fully automated approach to generate MTCs directly from the target program via functional coupling analysis. MR-Coupler first identifies functionally coupled method pairs based on signature commonality, function call, and state interaction. It then employs LLMs to generate concrete MTCs and refines them based on execution feedback. Finally, MR-Coupler amplifies and then validates the MTCs based on mutation analysis.

Our evaluation shows that MR-Coupler effectively generates valid MTCs for 98% of tasks and successfully reveals 22 confirmed bugs, improving valid MTC generation by 64.90%, and reducing false alarms by 36.56% compared to baselines. Moreover, MR-Coupler-generated MTCs achieve high consistency with human-written MR-skeletons, demonstrating MR-Coupler's potential to assist or even partially replace developers in constructing effective MTCs across diverse domains.

8 Data Availability

MR-Coupler and the experimental data are available at the website [57] and on Zenodo [58].

Acknowledgments

We would like to thank the anonymous reviewers for their insightful and constructive feedback. This work is supported by the National Natural Science Foundation of China (Grant No. 92582201), the Hong Kong SAR Research Grant Council (General Research Fund Ref: 16206524), and the Guangzhou HKUST Fok Ying Tung Research Institute.

References

[1] Alibaba. 2025. *Qwen3-coder.* Retrieved September 1, 2025 from https://qwenlm.github.io/blog/qwen3-coder/

[2] Simon Allier, Stéphane Vaucher, Bruno Dufour, and Houari A. Sahraoui. 2010. Deriving Coupling Metrics from Call Graphs. In *Tenth IEEE International Working Conference on Source Code Analysis and Manipulation, SCAM 2010, Timisoara, Romania, 12-13 September 2010.* IEEE Computer Society, 43–52. https://doi.org/10.1109/SCAM.2010.25

[3] Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE128 (June 2025), 23 pages. https://doi.org/10.1145/3729398

[4] Jon Ayerdi, Valerio Terragni, Aitor Arrieta, Paolo Tonella, Goiuria Sagardui, and Maite Arratibel. 2021. Generating metamorphic relations for cyber-physical systems with genetic programming: an industrial case study. In *Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering.* ACM, 1264–1274.

[5] Jon Ayerdi, Valerio Terragni, Gunel Jahangirova, Aitor Arrieta, and Paolo Tonella. 2024. GenMorph: Automatically Generating Metamorphic Relations via Genetic Programming. *IEEE Transactions on Software Engineering* (2024), 1–12.

[6] Arianna Blasi, Alessandra Gorla, Michael D. Ernst, Mauro Pezzè, and Antonio Carzaniga. 2021. MeMo: Automatically identifying metamorphic relations in Javadoc comments for test automation. *J. Syst. Softw.* 181 (2021), 111041. https://doi.org/10.1016/J.JSS.2021.111041

[7] Adam Bodicoat, Gunel Jahangirova, and Valerio Terragni. 2025. Understanding LLM-Driven Test Oracle Generation. In *2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware).* IEEE, 29–39.

[8] Jialun Cao, Meiziniu Li, Yeting Li, Ming Wen, Shing-Chi Cheung, and Haiming Chen. 2022. SemMT: A Semantic-Based Testing Approach for Machine Translation Systems. *ACM Transactions on Software Engineering and Methodology* 31, 2 (2022), 34e:1–34e:36.

[9] Jialun Cao, Wuqi Zhang, and Shing-Chi Cheung. 2024. Concerned with Data Contamination? Assessing Countermeasures in Code Language Model. *CoRR* abs/2403.16898 (2024). arXiv:2403.16898

[10] Songqiang Chen, Shuo Jin, and Xiaoyuan Xie. 2021. Testing Your Question Answering Software via Asking Recursively. In *International Conference on Automated Software Engineering.* IEEE, 104–116.

[11] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. 2018. Metamorphic Testing: A Review of Challenges and Opportunities. *ACM Comput. Surv.* 51, 1 (2018), 4:1–4:27. https://doi.org/10.1145/3143561

[12] Tsong Yueh Chen, Pak-Lok Poon, and Xiaoyuan Xie. 2016. METRIC: METamorphic Relation Identification based on the Category-choice framework. *J. Syst. Softw.* 116 (2016), 177–190. https://doi.org/10.1016/j.jss.2015.07.037

[13] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering.* 572–576.

[14] Steven Cho, Stefano Ruberto, and Valerio Terragni. 2025. LLMORPH: Automated Metamorphic Testing of Large Language Models. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering.* 4102–4105. https://doi.org/10.1109/ASE63991.2025.00385

[15] Steven Cho, Stefano Ruberto, and Valerio Terragni. 2025. Metamorphic Testing of Large Language Models for Natural Language Processing. In *Proceedings of the 41st IEEE International Conference on Software Maintenance and Evolution (ICSME).* IEEE, 174–186. https://doi.org/10.1109/ICSME64153.2025.00025

[16] DeepSeek. 2025. *DeepSeek-V3.1.* Retrieved September 1, 2025 from https://api-docs.deepseek.com/news/news250821

[17] Diennea. 2025. *SimplerPlannerTest.* https://github.com/diennea/herddb/blob/master/herddb-core/src/test/java/herddb/sql/SimplerPlannerTest.java

[18] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating Large Language Models in Class-Level Code Generation. In *nternational Conference on Software Engineering.* ACM, 81:1–81:13.

[19] Aryaz Eghbali and Michael Pradel. 2024. De-Hallucinator: Iterative Grounding for LLM-Based Code Completion. *CoRR* abs/2401.01701 (2024). arXiv:2401.01701

[20] Evalplus. 2025. *leaderboard.* Retrieved September 1, 2025 from https://evalplus.github.io/leaderboard.html

[21] FasterXML. 2025. *BasicParserFilteringTest.* https://github.com/FasterXML/jackson-core/blob/3.x/src/test/java/tools/jackson/core/unittest/filter/BasicParserFilteringTest.java#L432

[22] Enrico Fregnan, Tobias Baum, Fabio Palomba, and Alberto Bacchelli. 2019. A survey on software coupling relations and tools. *Inf. Softw. Technol.* 107 (2019), 159–178. https://doi.org/10.1016/J.INFSOF.2018.11.008

[23] Christoph Hazott and Daniel Große. 2025. LLM-assisted Metamorphic Testing of Embedded Graphics Libraries. In *Forum on Specification and Design Languages.* https://ics.jku.at/files/2025FDL_LLM-assisted_Metamorphic_Testing_of_Embedded_Graphics_Libraries.pdf

[24] Soneya Binta Hossain, Antonio Filieri, Matthew B. Dwyer, Sebastian G. Elbaum, and Willem Visser. 2023. Neural-Based Test Oracle Generation: A Large-Scale Evaluation and Lessons Learned. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023,*

San Francisco, CA, USA, December 3-9, 2023, Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.). ACM, 120–132. https://doi.org/10.1145/3611643.3616265

[25] Apache IoTDB. 2025. *IoTDB Issue #13691.* https://github.com/apache/iotdb/pull/13691

[26] JavaParser. 2024. *JavaParser.* Retrieved June 6, 2024 from https://javaparser.org/

[27] Jcabi. 2025. *GitHub Commit 777a078913.* https://github.com/jcabi/jcabi-github/commit/777a078913

[28] Yu Jiang, Jie Liang, Fuchen Ma, Yuanliang Chen, Chijin Zhou, Yuheng Shen, Zhiyong Wu, Jingzhou Fu, Mingzhe Wang, Shanshan Li, et al. 2024. When fuzzing meets llms: Challenges and opportunities. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 492–496.

[29] Knowledge Cutoff Information of GPT-4o-mini [n. d.]. https://community.openai.com/t/introducing-gpt-4o-mini-in-the-api/871594

[30] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler validation via equivalence modulo inputs. In *Conference on Programming Language Design and Implementation*. ACM, 216–226.

[31] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *International Conference on Software Engineering*. IEEE, 919–931.

[32] Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. 2023. Chain of Code: Reasoning with a Language Model-Augmented Code Emulator. *CoRR* abs/2312.04474 (2023). arXiv:2312.04474

[33] Jiapeng Li, Zheng Zheng, Yuning Xing, Daixu Ren, Steven Cho, and Valerio Terragni. 2025. MDPMORPH: An MDP-Based Metamorphic Testing Framework for Deep Reinforcement Learning Agents. In *Proceedings of the 36th IEEE International Symposium on Software Reliability Engineering*. 154–166. https://doi.org/10.1109/ISSRE66568.2025.00028

[34] Jiapeng Li, Zheng Zheng, Yuning Xing, Daixu Ren, Steven Cho, and Valerio Terragni. 2025. Metamorphic Testing of Deep Reinforcement Learning Agents with MDPMORPH. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*. 4086–4089. https://doi.org/10.1109/ASE63991.2025.00381

[35] Yujia Li, David H. Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, and et al. 2022. Competition-Level Code Generation with AlphaCode. *CoRR* abs/2203.07814 (2022). arXiv:2203.07814

[36] Huai Liu, Fei-Ching Kuo, Dave Towey, and Tsong Yueh Chen. 2014. How Effectively Does Metamorphic Testing Alleviate the Oracle Problem? *IEEE Transactions on Software Engineering* 40, 1 (2014), 4–22.

[37] LocationTech. 2025. *NTV2Test.* https://github.com/locationtech/proj4j/blob/master/core/src/test/java/org/locationtech/proj4j/datum/NTV2Test.java

[38] Quang-Hung Luu, Huai Liu, and Tsong Yueh Chen. 2023. Can ChatGPT Advance Software Testing Intelligence? An Experience Report on Metamorphic Testing. *CoRR* abs/2310.19204 (2023). arXiv:2310.19204 https://arxiv.org/abs/2310.19204

[39] Haoyang Ma, Qingchao Shen, Yongqiang Tian, Junjie Chen, and Shing-Chi Cheung. 2023. Fuzzing Deep Learning Compilers with HirGen. In *International Symposium on Software Testing and Analysis*. ACM, 248–260.

[40] Major. 2025. *Mutation Testing.* https://mutation-testing.org/

[41] Agustín Nolasco, Facundo Molina, Renzo Degiovanni, Alessandra Gorla, Diego Garbervetsky, Mike Papadakis, Sebastián Uchitel, Nazareno Aguirre, and Marcelo F. Frias. 2024. Abstraction-Aware Inference of Metamorphic Relations. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 450–472.

[42] OpenAI. 2025. *GPT-4o mini.* Retrieved September 1, 2025 from https://platform.openai.com/docs/models/gpt-4o-mini

[43] Optimatika. 2025. *OjAlgo Issue #49.* https://github.com/optimatika/ojAlgo/issues/49

[44] Optimatika. 2025. *OjAlgo Issue #49.* Retrieved September 1, 2025 from https://github.com/optimatika/ojAlgo/issues/49

[45] Denys Poshyvanyk, Andrian Marcus, Rudolf Ferenc, and Tibor Gyimóthy. 2009. Using information retrieval based coupling measures for impact analysis. *Empir. Softw. Eng.* 14, 1 (2009), 5–32. https://doi.org/10.1007/S10664-008-9088-2

[46] Ravin Ravi, Dylan Bradshaw, Stefano Ruberto, Gunel Jahangirova, and Valerio Terragni. 2025. LLMLOOP: Improving LLM-Generated Code and Tests through Automated Iterative Feedback Loops. In *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 930–934.

[47] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* 50, 1 (2023), 85–105.

[48] Sergio Segura, Gordon Fraser, Ana Belén Sánchez, and Antonio Ruiz Cortés. 2016. A Survey on Metamorphic Testing. *IEEE Trans. Software Eng.* 42, 9 (2016), 805–824. https://doi.org/10.1109/TSE.2016.2532875

[49] Sergio Segura, José Antonio Parejo, Javier Troya, and Antonio Ruiz Cortés. 2018. Metamorphic Testing of RESTful Web APIs. *IEEE Transactions on Software Engineering* 44, 11 (2018), 1083–1099.

[50] Seung Yeob Shin, Fabrizio Pastore, Domenico Bianculli, and Alexandra Baicoianu. 2024. Towards Generating Executable Metamorphic Relations Using Large Language Models. In *Quality of Information and Communications Technology - 17th International Conference on the Quality of Information and Communications Technology, QUATIC 2024, Pisa, Italy, September 11-13, 2024, Proceedings (Communications in Computer and Information Science, Vol. 2178)*, Antonia Bertolino,

João Pascoal Faria, Patricia Lago, and Laura Semini (Eds.). Springer, 126–141. https://doi.org/10.1007/978-3-031-70245-7_9

[51] Chengnian Sun, Vu Le, and Zhendong Su. 2016. Finding compiler bugs via live code mutation. In *International Conference on Object-Oriented Programming, Systems, Languages, and Applications,*. ACM, 849–863.

[52] Chang-Ai Sun, Yiqiang Liu, Zuoyi Wang, and W. K. Chan. 2016. μMT: a data mutation directed metamorphic relation acquisition methodology. In *International Workshop on Metamorphic Testing*. ACM, 12–18.

[53] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2024. ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation. *IEEE Transactions on Software Engineering* (2024), 1–19.

[54] Valerio Terragni, Gunel Jahangirova, Paolo Tonella, and Mauro Pezzè. 2020. Evolutionary Improvement of Assertion Oracles. In *Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1178–1189.

[55] Valerio Terragni, Annie Vella, Partha Roop, and Kelly Blincoe. 2025. The Future of AI-Driven Software Engineering. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (Jan. 2025). https://doi.org/10.1145/3715003

[56] TheAlgorithms. 2025. *AESEncryptionTest*. https://github.com/TheAlgorithms/Java/blob/master/src/test/java/com/thealgorithms/ciphers/AESEncryptionTest.java

[57] MR-Coupler. 2025. *MR-Coupler website*. Retrieved September 2, 2025 from https://mr-coupler.github.io/

[58] MR-Coupler. 2026. *MR-Coupler on Zenodo*. Retrieved April 2, 2026 from https://doi.org/10.5281/zenodo.19438045

[59] Christos Tsigkanos, Pooja Rani, Sebastian Müller, and Timo Kehrer. 2023. Variable Discovery with Large Language Models for Metamorphic Testing of Scientific Software. In *Computational Science - ICCS 2023 - 23rd International Conference, Prague, Czech Republic, July 3-5, 2023, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 14073)*. Springer, 321–335.

[60] vmgama. 2025. *dubbo*. https://github.com/vmgama/dubbo/blob/master/dubbo-common/src/main/java/org/apache/dubbo/common/io/Bytes.java

[61] Ying Wang, Bihuan Chen, Kaifeng Huang, Bowen Shi, Congying Xu, Xin Peng, Yijian Wu, and Yang Liu. 2020. An Empirical Study of Usages, Updates and Risks of Third-Party Libraries in Java Projects. In *International Conference on Software Maintenance and Evolution*. IEEE, 35–45.

[62] Brett Wooldridge. 2025. *SparseBitSet Issue #13*. https://github.com/brettwooldridge/SparseBitSet/issues/13

[63] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *International Conference on Software Engineering*. ACM, 126:1–126:13.

[64] Xiaoyuan Xie, Shuo Jin, and Songqiang Chen. 2023. qaAskeR$^+$: a novel testing method for question answering software via asking recursive questions. *Automated Software Engineering* 30, 1 (2023), 14.

[65] Xiaoyuan Xie, Shuo Jin, Songqiang Chen, and Shing-Chi Cheung. 2024. Word Closure-Based Metamorphic Testing for Machine Translation. *ACM Transactions on Software Engineering and Methodology* (jul 2024).

[66] Congying Xu, Songqiang Chen, Jiarong Wu, Shing-Chi Cheung, Valerio Terragni, Hengcheng Zhu, and Jialun Cao. 2024. MR-Adopt: Automatic Deduction of Input Transformation Function for Metamorphic Testing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024, Sacramento, CA, USA, October 27 - November 1, 2024*, Vladimir Filkov, Baishakhi Ray, and Minghui Zhou (Eds.). ACM, 557–569. https://doi.org/10.1145/3691620.3696020

[67] Congying Xu, Valerio Terragni, Hengcheng Zhu, Jiarong Wu, and Shing-Chi Cheung. 2024. MR-Scout: Automated Synthesis of Metamorphic Relations from Existing Test Cases. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 150.

[68] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. 2024. Exploring and Unleashing the Power of Large Language Models in Automated Code Translation. *CoRR* abs/2404.14646 (2024). arXiv:2404.14646

[69] Yuanyuan Yuan, Shuai Wang, Mingyue Jiang, and Tsong Yueh Chen. 2021. Perception Matters: Detecting Perception Failures of VQA Models Using Metamorphic Testing. In *Conference on Computer Vision and Pattern Recognition*. Computer Vision Foundation / IEEE, 16908–16917.

[70] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and Improving ChatGPT for Unit Test Generation. *Proc. ACM Softw. Eng.* 1, FSE (2024), 1703–1726. https://doi.org/10.1145/3660783

[71] Bo Zhang, Hongyu Zhang, Junjie Chen, Dan Hao, and Pablo Moscato. 2019. Automatic Discovery and Cleansing of Numerical Metamorphic Relations. In *IEEE International Conference on Software Maintenance and Evolution*. IEEE, 235–245.

[72] Jie Zhang, Junjie Chen, Dan Hao, Yingfei Xiong, Bing Xie, Lu Zhang, and Hong Mei. 2014. Search-based inference of polynomial metamorphic relations. In *ACM/IEEE International Conference on Automated Software Engineering*. ACM, 701–712.

[73] Jiaming Zhang, Chang-Ai Sun, Huai Liu, and Sijin Dong. 2025. Can Large Language Models Discover Metamorphic Relations? A Large-Scale Empirical Study. In *IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2025, Montreal, QC, Canada, March 4-7, 2025*. IEEE, 24–35. https://doi.org/10.1109/SANER64311.2025.00011

[74] Yifan Zhang, Tsong Yueh Chen, Matthew Pike, Dave Towey, Zhihao Ying, and Zhi Quan Zhou. 2025. Enhancing autonomous driving simulations: A hybrid metamorphic testing framework with metamorphic relations generated by GPT. *Inf. Softw. Technol.* 187 (2025), 107828. https://doi.org/10.1016/J.INFSOF.2025.107828

[75] Yifan Zhang, Dave Towey, and Matthew Pike. 2023. Automated Metamorphic-Relation Generation with ChatGPT: An Experience Report. In *47th IEEE Annual Computers, Software, and Applications Conference, COMPSAC 2023, Torino, Italy, June 26-30, 2023*. IEEE, 1780–1785. https://doi.org/10.1109/COMPSAC57700.2023.00275

[76] Yifan Zhang, Dave Towey, Matthew Pike, Quang-Hung Luu, Huai Liu, and Tsong Yueh Chen. 2025. Integrating Artificial Intelligence with Human Expertise: An In-depth Analysis of ChatGPT's Capabilities in Generating Metamorphic Relations. *CoRR* abs/2503.22141 (2025). arXiv:2503.22141 https://arxiv.org/abs/2503.22141

[77] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjun Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation. *Proc. ACM Softw. Eng.* 2, ISSTA (2025), 481–503. https://doi.org/10.1145/3728894

[78] Zhi Quan Zhou, Liqun Sun, Tsong Yueh Chen, and Dave Towey. 2020. Metamorphic Relations for Enhancing System Understanding and Use. *IEEE Transactions on Software Engineering* 46, 10 (2020), 1120–1154.

[79] Zingg. 2025. *Zingg Issue #60*. https://github.com/zinggAI/zingg/issues/60

Received 2026-02-25; accepted 2026-03-24